

Trabajo Final - Food Store

Sistema de Gestión de Pedidos de Comida

Se propone desarrollar un ecommerce llamado Food Store, orientado a la venta de productos de un negocio de comidas mediante una aplicación web full stack. El sistema permitirá gestionar categorías, productos y pedidos, y contará con una gestión de usuarios con dos perfiles principales: ADMIN, con acceso al panel de administración para realizar operaciones CRUD y gestionar el estado de los pedidos, y USUARIO, que podrá registrarse, iniciar sesión, navegar el catálogo, seleccionar productos, administrar un carrito y confirmar compras, además de consultar el historial y el estado de sus pedidos.

El desarrollo deberá realizarse tomando como guía el backlog de épicas e historias de usuario, de modo que cada funcionalidad implementada cumpla con sus criterios de aceptación y represente el comportamiento esperado del sistema. Esto implica transformar las historias en pantallas y endpoints concretos, incorporando validaciones, manejo de errores y respuestas correctas, para que el producto final sea navegable, verificable y refleje los flujos principales descritos en el TPI.

A partir de esta base, el desarrollo se abordará como un proyecto full stack guiado por un backlog Scrum, donde las funcionalidades estarán organizadas en épicas e historias de usuario. En la práctica, esto significa que el estudiante deberá transformar cada historia en pantallas del frontend (según el rol ADMIN o USUARIO) y en endpoints de la API REST del backend, respetando sus criterios de aceptación, validaciones y reglas de negocio. Por ejemplo, el rol ADMIN deberá contar con un panel para realizar operaciones CRUD sobre categorías y productos, además de gestionar pedidos y sus estados; mientras que el rol USUARIO deberá poder registrarse, iniciar sesión, navegar el catálogo, operar un carrito persistente, confirmar compras y consultar el historial/estado de sus pedidos. El objetivo es que el producto final sea navegable y verificable, y que los flujos principales del sistema puedan evaluarse claramente a partir de lo definido en las historias.

Sistema de Gestión de Pedidos de Comida	1
 Información del Proyecto	3
 Objetivos del Proyecto	3
 Estructura del Proyecto	3
Estructura Frontend (ejemplo de proyecto)	3
Estructura Backend (ejemplo de proyecto)	4
Sistema de Autenticación y Autorización	4
Roles y Permisos	5
Modelo UML	5

Funcionalidades por Módulo	6
1. Módulo de Autenticación	6
2. Módulo de Cliente - Store	7
3. Módulo de Cliente - Mis Pedidos	8
4. Módulo de Administración	9
 Diseño y UX	11
Pantallas de Referencia	11
Flujos de Usuario	16
Flujo de Compra (Cliente)	16
Flujo de Gestión de Producto (Admin)	16
Flujo de Gestión de Pedido (Admin)	17
Consideraciones Importantes	17
⚠ Seguridad	17
Historias de Usuario (Backlog Scrum)	17
Guía rápida: qué funcionalidades programar	17
 Entrega del Proyecto	18
Contenido de la Entrega	18
Método de Entrega	18
Anexo – Historias de Usuario	18
Historias de Usuario - Sistema de Gestión de Pedidos	18
Roles del Sistema	20
Épicas del Proyecto	21
EP-01: Gestion de Categorías	21
EP-02: Gestion de Usuarios	21
EP-03: Gestión de Productos	21
EP-04: Gestión de Pedidos	22
EP-05: Infraestructura y Arquitectura	22
Historias de Usuario por épica	22
EP-01: Gestion de Categorías	22
EP-02: Gestion de Usuarios	33
EP-03: Gestión de Productos	43
EP-04: Gestión de Pedidos	54
EP-05: Infraestructura y Arquitectura	67

Matriz de Trazabilidad	78
Historias de Usuario por Epica	78
Dependencias entre Historias	78
Priorización del Backlog	79
Sprint 1 - Fundamentos (Semana 1-2)	79
Sprint 2 - Usuarios y Productos (Semana 3-4)	80
Sprint 3 - Pedidos (Semana 5-6)	80
Definición de Completado (DoD)	80
Código	81
Testing	81
Documentación	81
Revisión	81
Funcionalidad	81



Información del Proyecto

Nombre: Food Store - Sistema de Gestión de Pedidos de Comida

Tecnologías:

- **Nombre:** Food Store - Sistema de Gestión de Pedidos de Comida
- **Tecnologías Frontend:** TypeScript, Vite, HTML5, CSS3
- **Tecnologías Backend:** JPA-HIBERNATE, Java 23, H2 (modo archivo)
- **Autenticación:** Gestión básica con localStorage (sólo fines educativos)



Objetivos del Proyecto

Desarrollar una aplicación web full stack completa para la gestión de un negocio de comidas, que permita:

1. **A los administradores:** Gestionar categorías, productos y pedidos
2. **A los clientes:** Navegar productos, realizar compras y seguir sus pedidos
3. **Sistema de carrito:** Funcional con persistencia en localStorage
4. **Backend por consola con persistencia.**

 Estructura del Proyecto**Estructura Frontend (ejemplo de proyecto)**

final-prog3/

```
| — index.html          # Redirección a login
| — package.json        # Dependencias y scripts
| — tsconfig.json       # Configuración TypeScript
| — vite.config.ts      # Configuración Vite
| — src/
|   | — main.ts         # Punto de entrada (vacío)
|   | — style.css       # Estilos globales
|   | — types/          # Definiciones de tipos TypeScript
|   |
|   | — utils/          # Utilidades y helpers
|   |
|   | — pages/          # Páginas de la aplicación |
|     | — auth/         # Autenticación
|     | — store/        # Páginas del cliente
|     |   | — home/
|     |   | — productDetail/
|     |   | — cart/
|     | — client/       # Área del cliente
|     |   | — orders/
|     | — admin/        # Panel de administración
```

Estructura Backend (ejemplo de proyecto)

foodstore-backend/

- |— build.gradle # Dependencias y configuración Gradle (Groovy)
- |— settings.gradle # Nombre del proyecto y módulos (si hubiera)
- |— gradlew # Wrapper (Linux/Mac) (opcional pero recomendado)
- |— gradlew.bat # Wrapper (Windows) (opcional pero recomendado)
- |— gradle/
- | └─ wrapper/
- | |— gradle-wrapper.jar
- | └─ gradle-wrapper.properties
- └─ src/
- └─ main/
- |— java/
- | └─ com/tp/jpa/
- | |— Main.java # Clase principal con el menú de consola
- | |— model/ # Entidades JPA
- | |— repository/ # Repositorios JPA
- └─ resources/
- └─ META-INF/persistence.xml # Configuración de la aplicación

Sistema de Autenticación y Autorización

Flujo de Autenticación

1. **Login:**
 - Usuario ingresa credenciales
 - Frontend valida credenciales son válidas con archivo json
 - Si es exitoso
 - Frontend guarda datos en localStorage
 - Redirecciona según el rol
2. **Validación de Sesión:**
 - Cada página protegida verifica localStorage
 - Si no hay sesión, redirecciona a login
 - Valida permisos según el rol
4. **Cierre de Sesión:**
 - Limpia localStorage
 - Redirecciona al login

Roles y Permisos

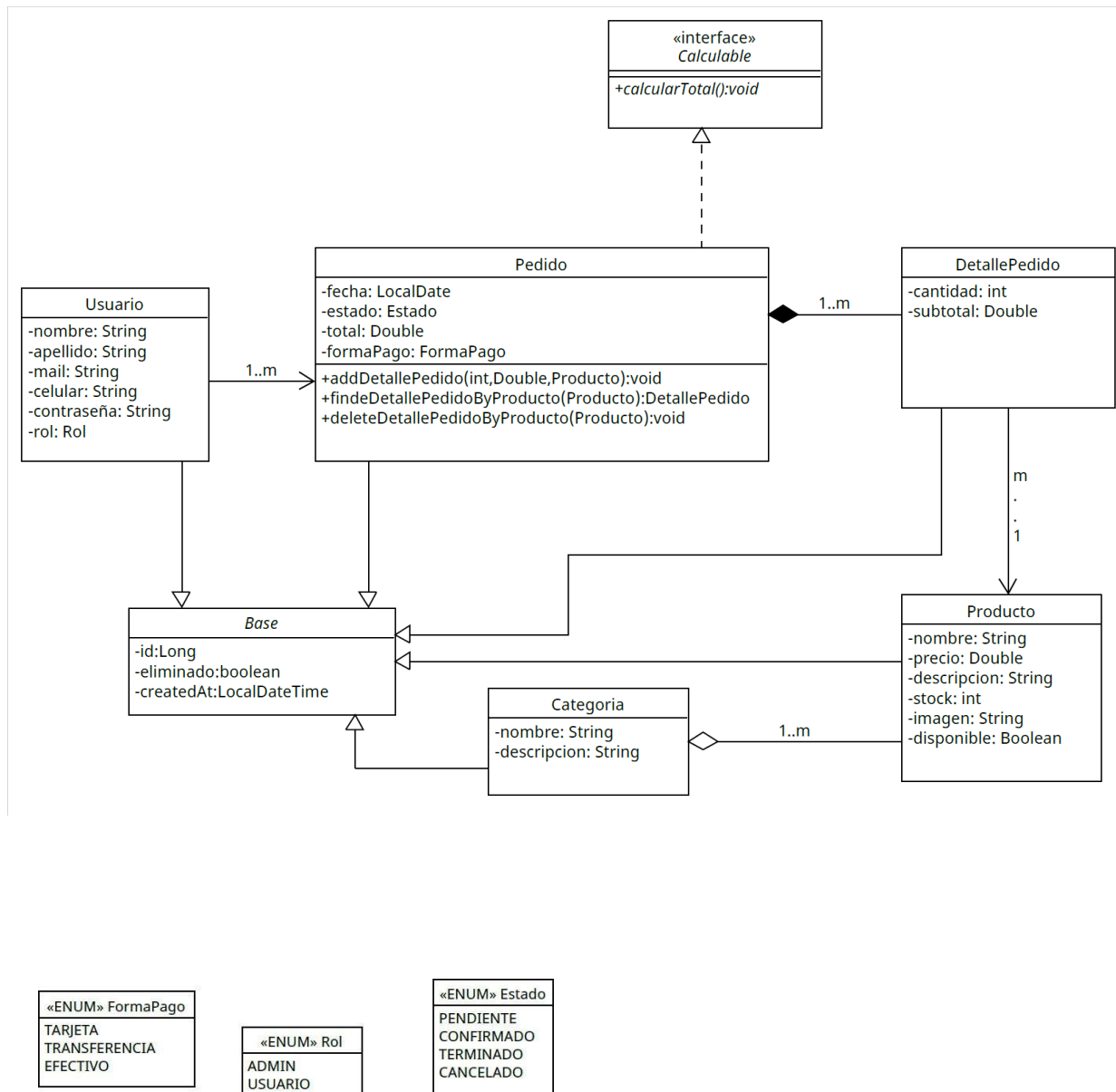
Administrador (Admin)

-  Acceso completo al panel de administración
-  Ver todas las categorías
-  Ver todos los producto
-  Ver todos los pedidos y filtrar por estado

Cliente (Usuario)

-  Ver catálogo de productos
-  Filtrar por categorías
-  Buscar productos
-  Ver detalle de productos
-  Agregar productos al carrito
-  Gestionar carrito (agregar, quitar, modificar cantidades)
-  Ver historial de pedidos propios
-  Ver detalle y estado de sus pedidos
-  NO tiene acceso al panel de administración

Modelo UML



Funcionalidades por Módulo

1. Módulo de Autenticación

Login (/src/pages/auth/login/)

Funcionalidad:

- Formulario con email y contraseña
- Validación de campos requeridos
- Validar con data/usuarios.json
- Manejo de errores de autenticación
- Redirección según rol del usuario

2. Módulo de Cliente - Store

Home / Catálogo (/src/pages/store/home/)

Características:

- Sidebar con categorías (data/categorias.json)
- Búsqueda en tiempo real
- Filtros:
 - Por categoría
 - Ordenamiento (nombre A-Z, Z-A, precio ascendente/descendente)
- Grid de productos (data/productos.json) con:
 - Imagen
 - Nombre
 - Descripción
 - Precio
 - Badge de disponibilidad
 - Click para ir al detalle
- Badge del carrito con cantidad de ítems
- Contador de productos encontrados
- Toggle del sidebar para mobile

Detalle de Producto (/src/pages/store/productDetail/)

Características:

- Buscar producto por id en data/productos.json
- Imagen grande del producto
- Información completa:
 - Nombre

- Descripción
 - Precio
 - Stock disponible
 - Estado (disponible/no disponible)
- Selector de cantidad con validación de stock
- Botón "Agregar al Carrito"
- Mensaje de confirmación
- Botón "Volver"
- Validaciones:
 - No permite agregar si no hay stock
 - No permite cantidad mayor al stock disponible
 - No permite agregar productos inactivos

Carrito de Compras (/src/pages/store/cart/)

Características:

- Lista de productos en el carrito con:
 - Imagen
 - Nombre
 - Precio unitario
 - Controles de cantidad (+/-)
 - Precio total por producto
 - Botón eliminar
- Resumen del pedido:
 - Subtotal
 - Total
- Botones:
 - "Vaciar Carrito"
- Validaciones:
 - Stock disponible al modificar cantidad
 - Campos requeridos en checkout
- Estado vacío con mensaje y botón a la tienda

Persistencia:

- Carrito guardado en localStorage
- Persiste entre sesiones
- Se limpia al confirmar al vaciar carrito

3. Módulo de Cliente - Mis Pedidos

Historial de Pedidos

(/src/pages/client/orders/) Características:

- Obtener de data/pedidos.json (solo pedidos del usuario)
- Lista de pedidos del usuario autenticado
- Tarjetas de pedido mostrando:
 - Número de pedido
 - Fecha y hora
 - Estado con badge de color
 - Resumen de productos (primeros 3 + contador)
 - Total del pedido
- Click en pedido abre modal con detalle completo
- Modal de detalle muestra:
 - Estado con icono visual
 - Información de entrega
 - Lista completa de productos
 - Desglose de costos (subtotal, envío, total)
 - Mensaje según estado del pedido
- Estado vacío si no hay pedidos

Estados de Pedido:

- `pending:`  Pendiente
- `processing:`  En Preparación
- `completed:`  Entregado
- `cancelled:`  Cancelado

4. Módulo de Administración

Dashboard

(/src/pages/admin/adminHome/)

Características:

- Sidebar de navegación administrativa
- 4 tarjetas con estadísticas:

- Total de categorías Obtener de data/categorias.json
- Total de productos Obtener de productos.json
- Total de pedidos Obtener de pedidos.json
- Productos disponibles
- Panel de resumen detallado:
 - Categorías activas
 - Productos activos/inactivos
 - Pedidos por estado
- Enlaces directos a cada módulo

Gestión de Categorías (/src/pages/admin/categories/)

Características:

- Obtener Categorías de data/categorias.json
- Tabla con todas las categorías:
 - ID
 - Imagen (thumbnail)
 - Nombre
 - Descripción

Gestión de Productos

(/src/pages/admin/products/) Características:

- Obtener de data/productos.json
- Tabla con todos los productos:
 - ID
 - Imagen (thumbnail)
 - Nombre
 - Descripción
 - Precio
 - Categoría (nombre)
 - Stock
 - Estado (disponible/no disponible)

Gestión de Pedidos Admin (/src/pages/admin/orders/)

Características:

- Obtener de data/pedidos.json (todos los pedidos)
- Filtro por estado de pedido
- Lista de TODOS los pedidos (no solo del usuario)
- Tarjetas de pedido mostrando:

- Número de pedido
 - Nombre del cliente
 - Fecha y hora
 - Estado con badge
 - Cantidad de productos
 - Total
- Ordenados por fecha (más recientes primero)



Diseño y UX

Pantallas de Referencia

Login

SuperFood
Iniciar Sesión

Email

ejemplo@correo.com

Contraseña

Ingresar tu contraseña

Ingresar

Cuentas de prueba:

Admin: admin@admin.com / 123456

Cliente: cliente@food.com / cliente123

Vista Home Store

Food Store

InicioMis PedidosCarritoJuan PérezCerrar Sesión

Categorías
Filtrar por categoría

Todos los productos

Hamburguesas

Pizzas

Empanadas

Bebidas

Papas Fritas

Buscar productos...

Ordenar por

Todos

Todos los Productos

19 productos

Hamburguesas

Hamburguesa Clásica

Carne 150g, lechuga, tomate, cebolla morada y mayonesa

4500.00

Disponible

Hamburguesas

Hamburguesa Doble

Doble carne 150g c/u, cheddar, panceta y salsa barbacoa

6200.00

Disponible

Hamburguesas

Hamburguesa Completa

Carne 150g, huevo, jamón, queso, lechuga y tomate

5500.00

Disponible

Hamburguesas

Hamburguesa Veggie

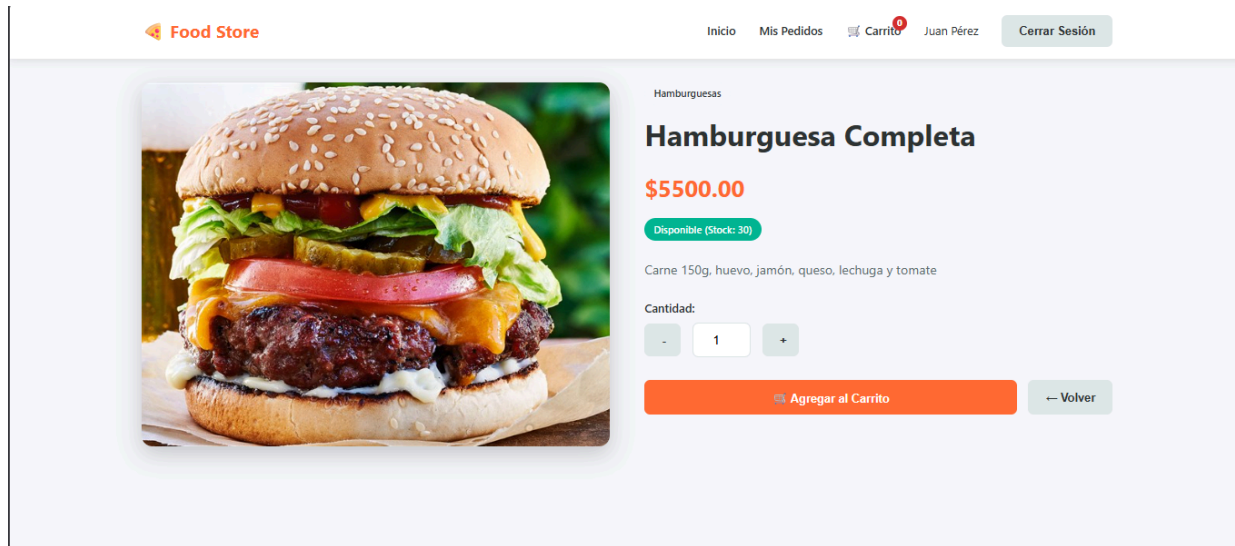
Medallón de lentejas y quinoa, rúcula, tomate confitado y hummus

5000.00

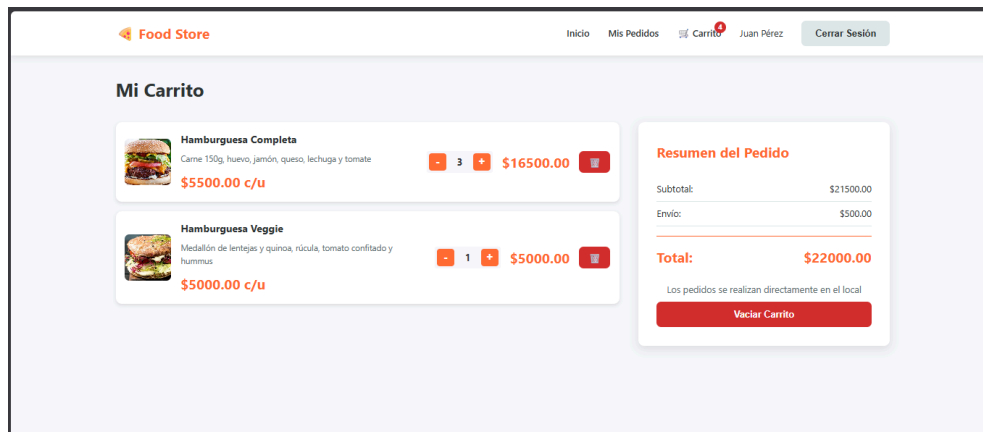
Disponible

Programación 3

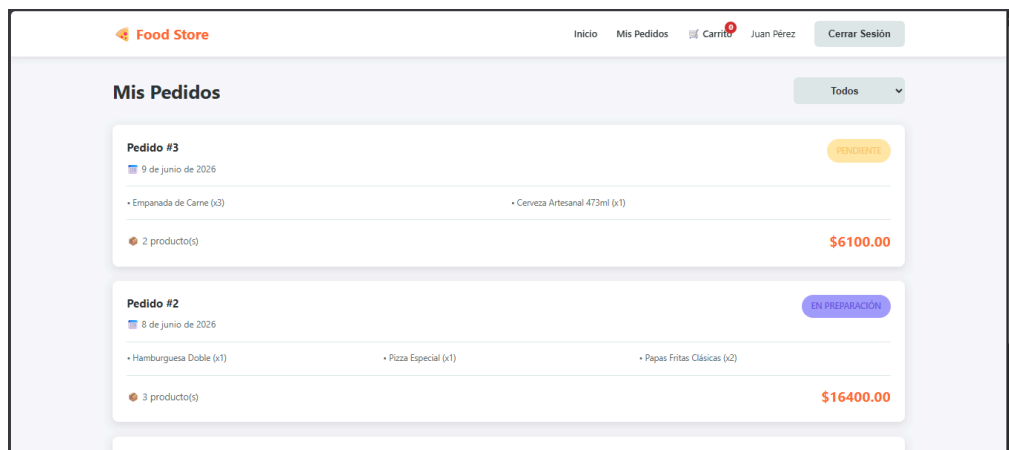
Vista Detalle de Producto

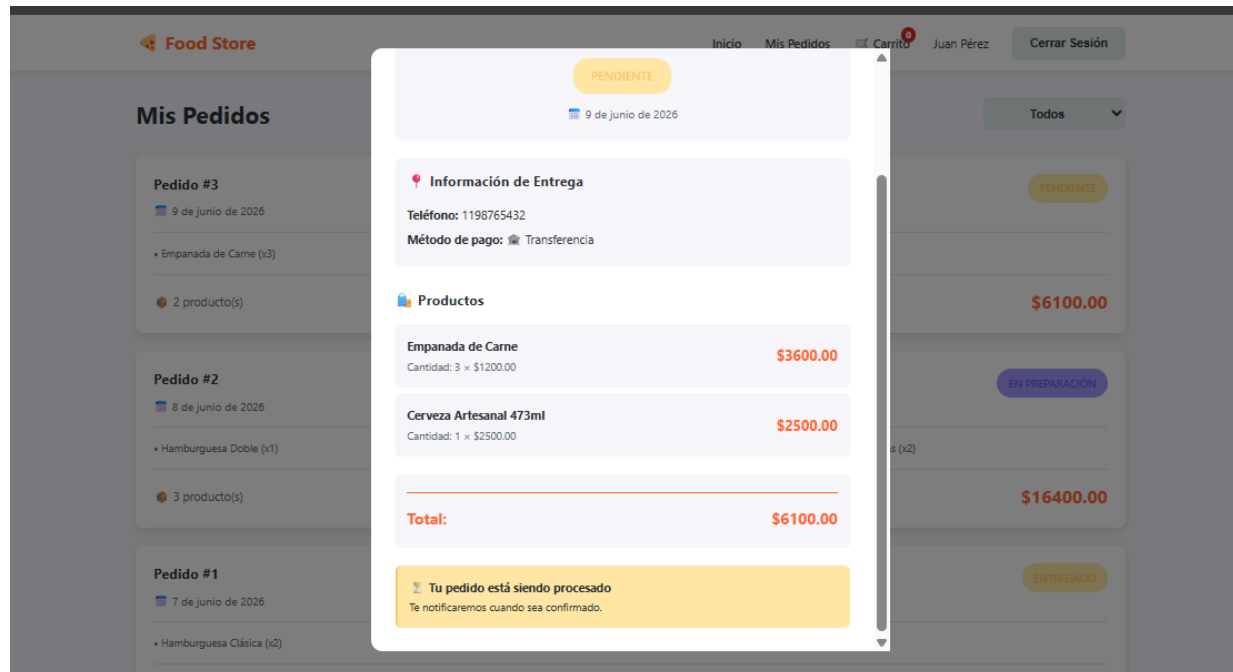


Vista de Carrito

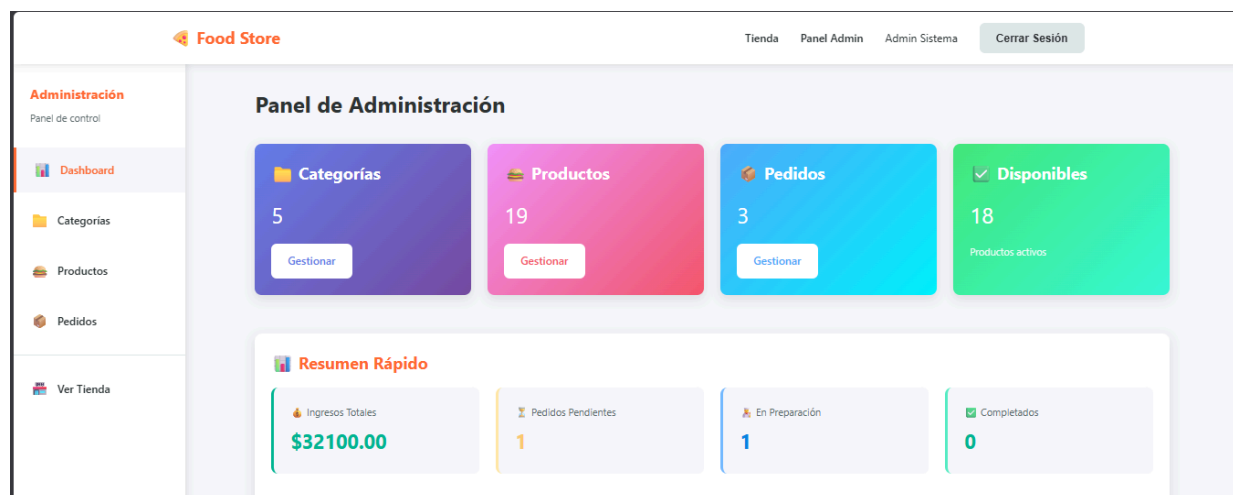


Pedidos del Cliente





Home Admin



Food Store

TiendaPanel AdminAdmin SistemaCerrar Sesión

Administración
Panel de control

Dashboard

Categorías

Productos

Pedidos

Ver Tienda

Gestión de Categorías

ID	Nombre	Descripción
1	Hamburguesas	Hamburguesas artesan...
2	Pizzas	Pizzas al horno de b...
3	Empanadas	Empanadas caseras ho...
4	Bebidas	Bebidas frías y cali...
5	Papas Fritas	Acompañamientos cruj...

Productos

Food Store

TiendaPanel AdminAdmin SistemaCerrar Sesión

Administración
Panel de control

Dashboard

Categorías

Productos

Pedidos

Ver Tienda

Gestión de Productos

ID	Imagen	Nombre	Descripción	Precio	Categoría	Stock	Estado
1		Hamburguesa Clásica	Carne 150g, lechuga, tomate, cebolla morada y mayonesa	\$4500.00	Hamburguesas	50	Disponible
2		Hamburguesa Doble	Doble carne 150g c/u, cheddar, panceta y salsa barbacoa	\$6200.00	Hamburguesas	40	Disponible
3		Hamburguesa Completa	Carne 150g, huevo, jamón, queso, lechuga y tomate	\$5500.00	Hamburguesas	30	Disponible
4		Hamburguesa Veggie	Medallón de lentejas y quinoa, rúcula, tomate confitado y hummus	\$5000.00	Hamburguesas	20	Disponible
5		Pizza Mozzarella	Mozzarella, salsa de tomate y orégano	\$3800.00	Pizzas	25	Disponible
6		Pizza Napolitana	Mozzarella, rodajas de tomate, ajo, aceitunas y albahaca	\$4500.00	Pizzas	25	Disponible

Food Store

TiendaPanel AdminAdmin SistemaCerrar Sesión

Administración

Panel de control

Dashboard

Categorías

Productos

Pedidos

Ver Tienda

Gestión de Pedidos

Todos los pedidos

Pedido #3

Cliente: Juan Pérez

9 de junio de 2026

2 producto(s)

PENDIENTE

\$6100.00

Pedido #2

Cliente: Juan Pérez

8 de junio de 2026

3 producto(s)

EN PREPARACIÓN

\$16400.00

Pedido #1

Cliente: Juan Pérez

7 de junio de 2026

2 producto(s)

ENTREGADO

\$9600.00

Flujos de Usuario

Flujo de Compra (Cliente)

1. Usuario se autentica
2. Navega por el catálogo (puede filtrar/buscar)
3. Click en producto → Ver detalle
4. Selecciona cantidad → Agregar al carrito
5. Continúa seleccionando o va al carrito
6. En el carrito: revisa productos, modifica cantidades

Flujo de Gestión de Producto (Admin)

1. Admin se autentica
2. Va a "Panel Admin" → "Productos"
3. Ve los productos en la tabla

Flujo de Gestión de Pedido (Admin)

1. Admin va a "Pedidos"
2. Ver lista de todos los pedidos
3. Puede filtrar por estado

Consideraciones Importantes

Seguridad

IMPORTANTE: Este proyecto NO implementa seguridad real:

- No hay tokens JWT
- La validación de rol y autenticación es solo frontend
- localStorage es fácilmente modificable
- **SOLO para fines educativos**

Backend JPA + Hibernate + H2 — Menú de Consola

1. Objetivo General

Implementar un sistema de gestión de pedidos de comida llamado **Food Store**, utilizando Java con JPA, Hibernate y base de datos H2 en archivo. La interacción con el sistema se realiza completamente a través de un menú de consola que permite gestionar todas las entidades del modelo: Categoría, Producto, Usuario y Pedido (con sus DetallePedido).

Nota: Quienes hayan aprobado el Parcial 2 ya cuentan con la base del proyecto (BaseRepository, CategoriaRepository, ProductoRepository y el menú de Categorías y Productos). Pueden partir de esa entrega. De todas formas, esta consigna describe en detalle todos los componentes del sistema,

incluyendo los ya desarrollados, para que cualquier alumno pueda construirlo desde cero.

2. Modelo de Dominio

El modelo se compone de una clase abstracta Base (anotada con `@MappedSuperclass`), una interfaz Calculable y cinco entidades JPA que extienden Base: Categoría, Producto, Usuario, Pedido y DetallePedido. A continuación se describe cada una y las relaciones entre ellas.

2.1 Entidades

Entidad	Descripción y campos propios
Base (abstracta)	Clase padre de todas las entidades. Define los campos comunes: id (Long, autogenerado con <code>@GeneratedValue</code>), eliminado (boolean, default false), createdAt (LocalDateTime). Anotada con <code>@MappedSuperclass</code> .
Calculable (interfaz)	Define el método <code>calcularTotal():void</code> . La clase Pedido lo implementa sumando los subtotales de sus DetallePedido y asignando el resultado a su propio campo total (no retorna valor; al ser void se invoca como <code>pedido.calcularTotal()</code> y luego se lee <code>pedido.getTotal()</code>).
Categoría	Representa una categoría de productos (ej: Hamburguesas, Bebidas). Campos: nombre (String), descripcion (String). Extiende Base.
Producto	Artículo del catálogo. Campos: nombre (String), precio (Double), descripcion (String), stock (int), imagen (String), disponible (Boolean). Extiende Base.
Usuario	Persona que opera el sistema o realiza pedidos. Campos: nombre (String), apellido (String), mail (String, único), celular (String), contrasena (String), rol (enum Rol). Extiende Base.
Pedido	Orden de compra. Campos: fecha (LocalDate), estado (enum Estado), total (Double), formaPago (enum FormaPago). Implementa Calculable. Extiende Base.
DetallePedido	Línea de un pedido. Representa un producto con su cantidad dentro de un pedido. Campos: cantidad (int), subtotal (Double), producto (<code>@ManyToOne</code> hacia Producto) y pedido (<code>@ManyToOne</code> hacia Pedido, FK del pedido al que pertenece). Extiende Base.

2.2 Relaciones entre entidades

Las relaciones hacia Categoría, Producto y Usuario son unidireccionales; la relación Pedido–DetallePedido es bidireccional (Pedido mantiene la colección con `mappedBy` y cada DetallePedido conoce a su Pedido). La siguiente tabla describe cada una con su tipo UML, cardinalidad, anotación JPA y comportamiento:

Tipo UML	Relación	Cardinalidad	Descripción y anotación JPA
Agregación unidireccional	Categoría → Producto	1 Categoría tiene muchos Productos	Producto conoce a su Categoría; Categoría no tiene lista de Productos. Producto declara: <code>@ManyToOne @JoinColumn(name="categoria_id") private Categoría categoria</code> . La categoría puede existir sin productos.
Composición bidireccional	Pedido → DetallePedido	1 Pedido tiene muchos DetallePedido	Relación bidireccional: los DetallePedido son creados y gestionados exclusivamente por Pedido a través del método <code>addDetallePedido()</code> , y cada DetallePedido conoce a su Pedido mediante la FK <code>pedido_id</code> . Pedido declara: <code>@OneToMany(mappedBy="pedido", cascade=CascadeType.ALL, orphanRemoval=true) private List<DetallePedido> detalles = new ArrayList<>()</code> . El valor de <code>mappedBy</code> ("pedido") debe coincidir exactamente con el nombre del campo <code>@ManyToOne</code> Pedido <code>pedido</code> declarado en DetallePedido. El <code>cascade</code> y <code>orphanRemoval</code> solo tienen efecto si el Pedido se eliminara físicamente; como en este sistema las bajas son siempre lógicas (<code>eliminado = true</code>), los DetallePedido permanecen en la BD.
Asociación unidireccional	DetallePedido → Producto	Muchos DetallePedido referencian 1 Producto	DetallePedido conoce al Producto que representa; Producto no conoce sus detalles. DetallePedido declara: <code>@ManyToOne @JoinColumn(name="producto_id") private Producto producto</code> . También declara la FK al pedido: <code>@ManyToOne @JoinColumn(name="pedido_id") private Pedido pedido</code> .
Asociación unidireccional	Pedido → Usuario	1 Usuario puede tener muchos Pedidos	Pedido conoce a su Usuario; Usuario no tiene lista de Pedidos. Pedido declara: <code>@ManyToOne @JoinColumn(name="usuario_id") private Usuario usuario</code> . Para obtener los pedidos de un usuario se usa la consulta JPQL en <code>PedidoRepository</code> (<code>buscarPorUsuario</code>).

Nota sobre `addDetallePedido()`: el método `addDetallePedido(int cantidad, Producto producto)` de la clase `Pedido` crea una instancia de `DetallePedido`, calcula su subtotal (`producto.getPrecio() * cantidad`), establece la referencia bidireccional `detalle.setPedido(this)`, y agrega el detalle a la colección

this.detalles. Es el único punto de creación de DetallePedido en el sistema. La colección detalles debe inicializarse en su declaración (= new ArrayList<>()) para evitar NullPointerException al invocar este método sobre un Pedido recién instanciado.

2.3 Enumerados

Enum	Valores
Rol	ADMIN, USUARIO
Estado	PENDIENTE, CONFIRMADO, TERMINADO, CANCELADO
FormaPago	TARJETA, TRANSFERENCIA, EFECTIVO

3. Estructura del Proyecto

El proyecto es Gradle con Java. Respetar la siguiente estructura de paquetes dentro de src/main/java/:

```
com.tp.jpa/
  model/                <- entidades JPA (Base, Categoria, Producto,
                        Usuario, Pedido, DetallePedido)
  model/enums/          <- enumerados (Rol, Estado, FormaPago)
  util/                 <- JPAUtil.java (fábrica de
                        EntityManagerFactory)
  repository/           <- repositorios (BaseRepository, y uno por
                        entidad)
  Main.java             <- clase principal con el menú de consola
```

La configuración de la base de datos se encuentra en src/main/resources/META-INF/persistence.xml. Se usa H2 en modo archivo (jdbc:h2:file:./data/jpa_db). Hibernate gestiona el esquema automáticamente con hbm2ddl.auto = update.

4. Componentes del Sistema

4.1 JPAUtil

Clase utilitaria que mantiene una única instancia de EntityManagerFactory para toda la aplicación. Se obtiene llamando a JPAUtil.getEntityManagerFactory(). El factory se cierra al finalizar la aplicación con JPAUtil.close().

4.2 BaseRepository<T>

Clase abstracta genérica que recibe la `Class<T>` de la entidad por constructor y obtiene el `EntityManagerFactory` desde `JPAUtil`. Implementa las operaciones CRUD comunes para todas las entidades. Cada método abre su propio `EntityManager` al inicio y lo cierra en un bloque `finally`.

Métodos que debe implementar:

Método	Comportamiento esperado
<code>guardar(T entity): T</code>	Abre <code>EntityManager</code> , inicia transacción y persiste la entidad: si su id es null usa <code>persist()</code> (alta, el ID lo asigna la BD), si ya tiene id usa <code>merge()</code> (actualización). Hace <code>commit</code> y retorna la entidad gestionada. En caso de excepción hace <code>rollback</code> . Cierra <code>EntityManager</code> en <code>finally</code> . IMPORTANTE: para mostrar el ID generado en un alta debe leerse del objeto retornado por <code>guardar()</code> (con <code>persist()</code> la propia instancia recibe el ID; si se optara por <code>merge()</code> el ID solo está en la copia devuelta, nunca en el objeto original).
<code>buscarPorId(Long id): Optional<T></code>	Abre <code>EntityManager</code> , busca con <code>find(clase, id)</code> . Si existe retorna <code>Optional.of(entidad)</code> , si no existe retorna <code>Optional.empty()</code> . Cierra <code>EntityManager</code> en <code>finally</code> .
<code>listarActivos(): List<T></code>	Abre <code>EntityManager</code> , ejecuta JPQL: <code>SELECT e FROM NombreEntidad e WHERE e.eliminado = false</code> . Retorna <code>List<T></code> . Cierra <code>EntityManager</code> en <code>finally</code> .
<code>eliminarLogico(Long id): boolean</code>	Abre <code>EntityManager</code> , busca la entidad por ID. Si existe, inicia transacción, establece <code>eliminado = true</code> , sincroniza con <code>merge()</code> (la entidad ya tiene id, por lo que <code>merge()</code> es correcto), hace <code>commit</code> y retorna <code>true</code> . Si no existe retorna <code>false</code> . Cierra <code>EntityManager</code> en <code>finally</code> .

Nota técnica: la consulta JPQL de `listarActivos()` debe construirse usando el nombre simple de la clase (`getEntityClass().getSimpleName()`) para que funcione correctamente con todas las entidades que extienden `Base`.

4.3 CategoriaRepository

Extiende `BaseRepository<Categoria>`. El constructor llama a `super(Categoria.class)`. No requiere métodos adicionales: hereda todo el CRUD del repositorio base.

4.4 ProductoRepository

Extiende `BaseRepository<Producto>`. El constructor llama a `super(Producto.class)`. Además del CRUD heredado implementa:

buscarPorCategoria(Long categoriaId): List<Producto>

Retorna los productos activos que pertenecen a la categoría indicada. Debe usar JPQL con parámetro nombrado, filtrar por `eliminado = false`, y retornar un `List<Producto>` sin casteos manuales. Incluir

comentario explicando la consulta.

```
// Consulta JPQL: retorna productos activos de una categoría específica
// Filtra por categoria.id y por eliminado = false para excluir bajas lógicas
String jpql = "SELECT p FROM Categoria c JOIN c.productos p WHERE c.id = :catId AND p.eliminado = false";
List<Producto> q = em.createQuery(jpql, Producto.class)
    .setParameter("catId", categoriaId)
    .getResultList();
```

4.5 UsuarioRepository

Extiende BaseRepository<Usuario>. El constructor llama a super(Usuario.class). Además del CRUD heredado implementa:

buscarPorMail(String mail): Optional<Usuario>

Retorna el usuario activo con el mail indicado. Usar JPQL con parámetro nombrado, filtrar por eliminado = false. Retornar Optional.of(usuario) si existe, Optional.empty() si no. Incluir comentario explicando la consulta.

```
// Consulta JPQL: busca un usuario activo por su dirección de correo electrónico
// Retorna Optional para manejar el caso en que el mail no esté registrado
String jpql = "SELECT u FROM Usuario u WHERE u.mail = :mail AND u.eliminado = false";
List<Usuario> q = em.createQuery(jpql, Usuario.class)
    .setParameter("mail", mail);
    .getResultList();
```

4.6 PedidoRepository

Extiende BaseRepository<Pedido>. El constructor llama a super(Pedido.class). Además del CRUD heredado implementa:

buscarPorUsuario(Long idUsuario): List<Pedido>

Retorna los pedidos activos del usuario indicado. Usar JPQL con parámetro nombrado, filtrar por eliminado = false.

```
// Consulta JPQL: retorna todos los pedidos activos de un usuario dado su ID
// Filtra por eliminado = false para excluir pedidos dados de baja lógica
String jpql = "SELECT p FROM Usuario u JOIN u.pedidos p WHERE u.id = :uid AND p.eliminado = false";
List<Pedido> q = em.createQuery(jpql, Pedido.class)
```

```
.setParameter("uid", idUsuario)
.getResultList();
```

buscarPorEstado(Estado estado): List<Pedido>

Retorna los pedidos activos que coincidan con el estado indicado. Usar JPQL con parámetro nombrado, filtrar por eliminado = false.

```
// Consulta JPQL: retorna todos los pedidos activos con un estado
// específico
// Útil para filtrar PENDIENTE, CONFIRMADO, TERMINADO o CANCELADO
String jpql = "SELECT p FROM Pedido p WHERE p.estado = :estado AND
p.eliminado = false";
List<Pedido> q = em.createQuery(jpql, Pedido.class)
.setParameter("estado", estado)
.getResultList();
```

5. Menú de Consola

La clase Main presenta un menú principal con las siguientes secciones. El orden de uso natural es: primero crear Categorías, luego Productos (asociándolos a una categoría), luego Usuarios, y por último Pedidos.

Opción del menú principal	Descripción
1. Gestionar Categorías	ABM completo de categorías.
2. Gestionar Productos	ABM completo de productos, asociados a una categoría.
3. Gestionar Usuarios	ABM completo de usuarios con búsqueda por mail.
4. Gestionar Pedidos	Alta de pedido con detalles, cambio de estado, baja y listados.
5. Reportes	Consultas JPQL: productos por categoría, pedidos por usuario, pedidos por estado, total facturado.
0. Salir	Cierra el EntityManagerFactory y termina la aplicación.

5.1 Submenú Categorías

Este submenú es el punto de entrada obligatorio del sistema. Antes de crear productos es necesario tener al menos una categoría activa.

Opción	Comportamiento
1. Alta	Solicitar nombre (obligatorio, no puede estar vacío) y descripción (opcional). Crear instancia de Categoría, llamar a categoriaRepo.guardar(). Mostrar el ID generado por la BD.
2. Modificar	Listar categorías activas con listarActivos(). Solicitar ID. Si no corresponde a una categoría activa, mostrar error y volver al menú. Mostrar valores actuales. Solicitar nuevo nombre y descripción: si el usuario deja el campo vacío (presiona Enter sin escribir), conservar el valor anterior. Llamar a guardar() con la entidad modificada.
3. Baja lógica	Solicitar ID. Llamar a eliminarLogico(id). Si retorna false, mostrar error (no encontrado o ya dado de baja). Si retorna true, confirmar mostrando el nombre de la categoría afectada.
4. Listado	Llamar a listarActivos() y mostrar todas las categorías activas con: ID, nombre y descripción.
0. Volver	Retornar al menú principal.

5.2 Submenú Productos

Los productos deben estar asociados a una categoría. Si no hay categorías activas al dar de alta un producto, se informa y se cancela la operación.

Opción	Comportamiento
1. Alta	Listar categorías activas. Si no hay ninguna, mostrar error y volver. Solicitar selección de categoría por ID. Solicitar: nombre (obligatorio), descripción, precio (Double, mayor a 0), stock (int, mayor o igual a 0), imagen (opcional), disponible (S/N, default true). Validar precio y stock: si son inválidos, mostrar error y no persistir. Crear instancia de Producto con la Categoría seleccionada. Llamar a productoRepo.guardar(). Mostrar ID generado y categoría asignada.
2. Modificar	Listar productos activos con listarActivos(). Solicitar ID. Si no existe o está dado de baja, mostrar error. Mostrar valores actuales. Solicitar nuevos valores para nombre, precio y stock: campo vacío conserva valor anterior. Validar precio > 0 y stock >= 0 si se ingresa un nuevo valor. Guardar cambios.
3. Baja lógica	Solicitar ID. Llamar a eliminarLogico(id). Si retorna false, mostrar error. Si retorna true, confirmar mostrando el nombre del producto afectado.
4. Listado	Llamar a listarActivos() y mostrar: ID, nombre, precio, stock, estado de disponibilidad y nombre de su categoría.
0. Volver	Retornar al menú principal.

5.3 Submenú Usuarios

Los usuarios son necesarios para generar pedidos. Se recomienda crear al menos un usuario antes de

acceder al menú de Pedidos.

Opción	Comportamiento
1. Alta	Solicitar: nombre, apellido, mail, celular (opcional), contraseña y rol (mostrar opciones: ADMIN / USUARIO). Validar que el mail no esté ya en uso llamando a <code>usuarioRepo.buscarPorMail(mail)</code> : si existe usuario activo con ese mail, mostrar error y no persistir. Crear instancia de Usuario. Llamar a <code>usuarioRepo.guardar()</code> . Mostrar ID generado.
2. Modificar	Listar usuarios activos. Solicitar ID. Si no existe o está dado de baja, mostrar error. Mostrar valores actuales. Solicitar nuevos valores para nombre, apellido, celular y contraseña: campo vacío conserva valor anterior. Si se cambia el mail, validar que no esté en uso por otro usuario (<code>buscarPorMail</code> y verificar que el ID sea distinto). Guardar cambios.
3. Baja lógica	Solicitar ID. Llamar a <code>eliminarLogico(id)</code> . Si retorna false, mostrar error. Si retorna true, confirmar mostrando nombre y apellido del usuario. Sus pedidos permanecen en el sistema.
4. Listado	Llamar a <code>listarActivos()</code> y mostrar: ID, nombre completo, mail y rol.
5. Buscar por mail	Solicitar mail. Llamar a <code>usuarioRepo.buscarPorMail(mail)</code> . Si el Optional contiene un usuario, mostrar todos sus datos (sin mostrar la contraseña). Si está vacío, informar que no existe usuario activo con ese mail.
0. Volver	Retornar al menú principal.

5.4 Submenú Pedidos

El alta de pedido es la operación más compleja del sistema. Involucra validaciones de stock, cálculo de subtotales, reducción de inventario y debe ejecutarse dentro de una única transacción atómica.

Opción	Comportamiento
1. Alta de pedido	Ver detalle completo en sección 5.4.1.
2. Cambiar estado	Solicitar ID de pedido. Si no existe o está dado de baja, mostrar error. Mostrar estado actual. Mostrar opciones: PENDIENTE, CONFIRMADO, TERMINADO, CANCELADO. Actualizar campo estado en la entidad. Llamar a <code>pedidoRepo.guardar()</code> . Confirmar mostrando ID y nuevo estado.
3. Baja lógica	Solicitar ID. Llamar a <code>eliminarLogico(id)</code> . Si retorna false, mostrar error. Si retorna true, confirmar mostrando ID y total del pedido. El stock de los productos NO se restaura. Los <code>DetallePedido</code> permanecen en la BD.
4. Listado	Llamar a <code>listarActivos()</code> y mostrar para cada pedido: ID, fecha, estado, forma de pago, nombre del usuario y total.
5. Pedidos por	Listar usuarios activos, solicitar selección. Llamar a

usuario	pedidoRepo.buscarPorUsuario(idUsuario). Mostrar: ID, fecha, estado y total de cada pedido. Si la lista está vacía, informarlo.
6. Pedidos por estado	Mostrar los estados disponibles, solicitar selección. Llamar a pedidoRepo.buscarPorEstado(estado). Mostrar: ID, fecha, nombre de usuario y total. Si la lista está vacía, informarlo.
0. Volver	Retornar al menú principal.

5.4.1 Alta de Pedido — Flujo detallado

Este flujo debe ejecutarse dentro de una única transacción. Si falla cualquier validación, se hace rollback completo y no se modifica nada en la base de datos.

- Listar usuarios activos y solicitar al operador que seleccione uno por ID. Si no existen usuarios activos, informar y cancelar.
- Solicitar la forma de pago mostrando las opciones del enum FormaPago: TARJETA, TRANSFERENCIA, EFECTIVO.
- Ingresar productos al pedido (ciclo repetible):
 - Mostrar el catálogo de productos activos con ID, nombre, precio y stock disponible.
 - Solicitar ID del producto a agregar. Si no existe o está dado de baja, mostrar error.
 - Verificar que el producto tenga disponible = true. Si no, mostrar error y no agregar.
 - Solicitar cantidad (entero mayor a 0). Verificar que el stock del producto sea suficiente. Si no alcanza, mostrar el stock disponible e informar el error.
 - Si las validaciones pasan, agregar el producto y la cantidad a una lista temporal en memoria (aún no se persiste nada).
 - Preguntar si desea agregar otro producto. Repetir hasta que el operador indique que no.
- Si la lista de productos está vacía al confirmar, informar que el pedido debe tener al menos un detalle y cancelar.
- Una vez confirmada la lista, abrir un único EntityManager e iniciar una sola transacción (todo el alta ocurre con este mismo EntityManager; la lista temporal previa solo guarda el ID del producto y la cantidad, no entidades de otro EntityManager):
 - Crear la instancia de Pedido con el usuario, fecha actual (LocalDate.now()), estado PENDIENTE, forma de pago seleccionada.
 - Por cada producto en la lista temporal: recuperar el Producto gestionado con em.find(Producto.class, idProducto) dentro de este EntityManager, crear un DetallePedido con la cantidad, calcular subtotal = producto.getPrecio() * cantidad, y asociarlo al Pedido mediante pedido.addDetallePedido(...).
 - Llamar a pedido.calcularTotal() (suma de subtotales de todos los detalles).
 - Reducir el stock de cada producto: producto.setStock(producto.getStock() - cantidad). Como el Producto fue recuperado con em.find() en este mismo EntityManager está gestionado, por lo que el cambio se sincroniza automáticamente al hacer commit (no hace falta merge() explícito).
 - Persistir el Pedido nuevo con em.persist(pedido); por el cascade = CascadeType.ALL los DetallePedido asociados se persisten en la misma operación.

- Hacer commit.
- Si ocurre cualquier excepción durante la transacción, hacer rollback completo. Mostrar mensaje de error.
- Al completar exitosamente, mostrar: ID generado, fecha, usuario, forma de pago, listado de productos con cantidades y subtotales, y el total del pedido.

5.5 Submenú Reportes

Opción	Comportamiento
1. Productos por categoría	Listar categorías activas y solicitar selección. Llamar a <code>productoRepo.buscarPorCategoria(idCategoria)</code> . Mostrar: ID, nombre, precio y stock de cada producto. Si no hay productos activos en esa categoría, informarlo explícitamente.
2. Pedidos por usuario	Listar usuarios activos y solicitar selección. Llamar a <code>pedidoRepo.buscarPorUsuario(idUsuario)</code> . Mostrar: ID, fecha, estado, forma de pago y total de cada pedido. Si no hay pedidos, informarlo.
3. Pedidos por estado	Mostrar opciones del enum Estado y solicitar selección. Llamar a <code>pedidoRepo.buscarPorEstado(estado)</code> . Mostrar: ID, fecha, nombre de usuario y total. Si no hay pedidos con ese estado, informarlo.
4. Total facturado	Obtener los pedidos con estado TERMINADO llamando a <code>pedidoRepo.buscarPorEstado(Estado.TERMINADO)</code> . Sumar los campos total de todos los pedidos del resultado (por ejemplo con <code>mapToDouble(Pedido::getTotal).sum()</code> , considerando null como 0). Mostrar el resultado formateado a dos decimales con <code>String.format(Locale.US, "%.2f", total)</code> para evitar el error de representación propio de Double (ej: Total facturado: \$12500.00). Si no hay pedidos terminados, mostrar \$0.00.
0. Volver	Retornar al menú principal.

6. Reglas Técnicas

- Cada método de repositorio debe abrir su propio EntityManager al inicio y cerrarlo en un bloque finally, independientemente de si la operación fue exitosa o no.
- Las operaciones de escritura (guardar, eliminarLogico) deben manejar la transacción con `begin()` / `commit()` y `rollback()` en caso de excepción.
- El alta de pedido debe ejecutarse en una única transacción: si cualquier validación falla después de iniciada, se hace rollback completo y no se modifica ningún dato en la BD.
- Los métodos con JPQL personalizado deben incluir un comentario que explique qué hace la consulta.
- Las bajas son siempre lógicas: `eliminado = true`. El registro permanece en la BD y no debe aparecer en ningún listado activo.

- Dejar un campo en blanco durante una modificación conserva el valor anterior (no se pisa con null ni con cadena vacía).
- Al finalizar la aplicación (opción 0 del menú principal), llamar a JPAUtil.close() para cerrar el EntityManagerFactory correctamente.

7. Historias de Usuario — Backend

7.1 Repositorios

HU-01 BaseRepository con CRUD genérico
Como: desarrollador
Quiero: contar con un BaseRepository<T> que implemente las operaciones CRUD comunes
Para: no repetir código de persistencia en cada repositorio específico
Prioridad: Alta
Story Points: 18

Criterios de Aceptación

1. guardar() abre su propia transacción; usa persist() si el id es null (alta) o merge() si ya tiene id (actualización), y hace commit. Hace rollback ante excepción. Retorna la entidad gestionada, de la cual debe leerse el ID generado.
2. buscarPorId() retorna Optional<T>: Optional.of(entidad) si existe, Optional.empty() si no.
3. listarActivos() usa JPQL con WHERE e.eliminado = false y retorna List<T>.
4. eliminarLogico() busca por ID, establece eliminado = true, persiste y retorna true. Retorna false si no encuentra el registro.
5. Cada método cierra el EntityManager en un bloque finally.

HU-02 CategoriaRepository y ProductoRepository
Como: desarrollador
Quiero: contar con CategoriaRepository y ProductoRepository que extiendan BaseRepository
Para: operar sobre cada entidad sin reescribir el CRUD base
Prioridad: Alta

Story Points: 12

Criterios de Aceptación

1. CategoriaRepository extiende BaseRepository<Categoria> y llama a super(Categoria.class).
2. ProductoRepository extiende BaseRepository<Producto> y llama a super(Producto.class).
3. ProductoRepository incluye buscarPorCategoria(Long categoriaId) con JPQL tipado.
4. La consulta JPQL filtra por categoria.id = :catId y eliminado = false.
5. El método retorna List<Producto>.
6. El método incluye un comentario explicando la consulta JPQL.

HU-03 | UsuarioRepository con buscarPorMail

Como: desarrollador

Quiero: contar con UsuarioRepository que extienda BaseRepository<Usuario> y provea buscarPorMail()

Para: validar unicidad de mail y buscar usuarios sin reescribir el CRUD base

Prioridad: Alta

Story Points: 8

Criterios de Aceptación

1. UsuarioRepository extiende BaseRepository<Usuario> y llama a super(Usuario.class).
2. buscarPorMail() usa JPQL con parámetro nombrado :mail y WHERE e.eliminado = false.
3. Retorna Optional<Usuario>: Optional.of(usuario) si existe, Optional.empty() si no.
4. La consulta se ejecuta con TypedQuery<Usuario> y devuelve el resultado mediante getResultList().
5. El método cierra el EntityManager en un bloque finally.
6. Incluye un comentario explicando la consulta JPQL.

HU-04 | PedidoRepository con consultas JPQL

Como: desarrollador

Quiero: contar con PedidoRepository con buscarPorUsuario() y buscarPorEstado()

Para: obtener pedidos filtrados sin escribir JPQL fuera del repositorio

Prioridad: Alta

Story Points: 10

Criterios de Aceptación

1. PedidoRepository extiende BaseRepository<Pedido> y llama a super(Pedido.class).
2. buscarPorUsuario() filtra por usuario.id = :uid y eliminado = false. Retorna List<Pedido>.
3. buscarPorEstado() filtra por estado = :estado y eliminado = false. Retorna List<Pedido>.
4. Ambos usan List<Pedido>.
5. Cada método cierra el EntityManager en un bloque finally.
6. Cada método incluye un comentario explicando la consulta JPQL.

7.2 Categorías

HU-05 | Dar de alta una categoría

Como: operador del sistema

Quiero: crear una nueva categoría ingresando nombre y descripción

Para: organizar los productos del catálogo en grupos temáticos

Prioridad: Alta

Story Points: 8

Criterios de Aceptación

1. El sistema solicita nombre y descripción por consola.
2. Si el nombre está vacío, el sistema informa el error y no persiste.
3. Al guardar exitosamente, se muestra el ID generado por la base de datos.
4. La categoría queda con eliminado = false.

HU-06 | Modificar una categoría existente

Como: operador del sistema

Quiero: editar el nombre o la descripción de una categoría ya creada

Para: corregir errores sin tener que borrar y recrear el registro

Prioridad: Alta

Story Points: 10

Criterios de Aceptación

1. El sistema lista las categorías activas antes de pedir el ID.
2. Si el ID no corresponde a ninguna categoría activa, se muestra un mensaje de error.
3. Se muestran los valores actuales antes de pedir los nuevos.
4. Dejar un campo en blanco mantiene el valor anterior.
5. El cambio se persiste correctamente en la base de datos.

HU-07 | Dar de baja una categoría

Como: operador del sistema

Quiero: dar de baja una categoría que ya no se utiliza

Para: ocultarla del sistema sin perder el historial de datos

Prioridad: Alta

Story Points: 7

Criterios de Aceptación

1. La baja es lógica: se establece eliminado = true, el registro permanece en la BD.
2. Si el ID no existe o ya está dado de baja, el sistema informa el error.
3. La categoría dada de baja no aparece en ningún listado activo.
4. Se confirma la operación mostrando el nombre de la categoría afectada.

7.3 Productos

HU-08 | Dar de alta un producto

Como: operador del sistema

Quiero: registrar un nuevo producto asociándolo a una categoría existente

Para: incorporar artículos al catálogo con toda su información básica

Prioridad: Alta

Story Points: 12

Criterios de Aceptación

1. El sistema lista las categorías activas para que el operador seleccione una.
2. Si no hay categorías activas, se informa y se cancela la operación.
3. Se solicitan: nombre (obligatorio), descripción, precio (mayor a 0), stock (mayor o igual a 0), imagen y disponible.
4. Si precio o stock tienen valores inválidos, se informa el error y no se persiste.
5. Al guardar se muestra el ID generado y la categoría asignada.

HU-09 | Modificar un producto

Como: operador del sistema

Quiero: actualizar el nombre, precio y stock de un producto existente

Para: mantener el catálogo actualizado sin recrear el registro

Prioridad: Alta

Story Points: 10

Criterios de Aceptación

1. El sistema lista los productos activos antes de pedir el ID.
2. Si el ID no existe o el producto está dado de baja, se muestra error.
3. Se muestran los valores actuales antes de pedir los nuevos.

- | |
|---|
| 4. Dejar un campo en blanco conserva el valor anterior. |
| 5. Precio no puede actualizarse a un valor menor o igual a 0. |
| 6. Stock no puede actualizarse a un valor negativo. |

HU-10 | Dar de baja un producto

Como: operador del sistema

Quiero: dar de baja un producto que ya no está disponible

Para: retirarlo del catálogo activo sin eliminar su historial

Prioridad: Alta

Story Points: 8

Criterios de Aceptación

- | |
|--|
| 1. La baja es lógica: eliminado = true, el registro permanece en la BD. |
| 2. Si el ID no existe o ya está dado de baja, se informa el error. |
| 3. El producto dado de baja no aparece en el listado de productos activos. |
| 4. Se muestra confirmación con el nombre del producto afectado. |

HU-11 | Consulta JPQL: Productos por categoría

Como: operador del sistema

Quiero: ver todos los productos activos que pertenecen a una categoría específica

Para: consultar el catálogo filtrado sin revisar todos los productos

Prioridad: Alta

Story Points: 10

Criterios de Aceptación

- | |
|--|
| 1. El sistema lista las categorías activas para que el operador seleccione una. |
| 2. La consulta está implementada en ProductoRepository con JPQL y parámetro nombrado :catId. |

- | |
|---|
| 3. El método retorna List<Producto> sin casteos manuales. |
| 4. Solo se incluyen productos con eliminado = false. |
| 5. El resultado muestra: ID, nombre, precio y stock de cada producto. |
| 6. Si la categoría no tiene productos activos, se informa explícitamente. |

7.4 Usuarios

HU-12 | Dar de alta un usuario

Como: operador del sistema

Quiero: registrar un nuevo usuario con sus datos y rol

Para: que pueda ser asociado a pedidos en el sistema

Prioridad: Alta

Story Points: 8

Criterios de Aceptación

- | |
|--|
| 1. Se solicitan: nombre, apellido, mail, celular, contraseña y rol (ADMIN / USUARIO). |
| 2. Si el mail ya está en uso (buscarPorMail retorna un Optional no vacío), se informa el error y no se persiste. |
| 3. Al guardar exitosamente se muestra el ID generado. |
| 4. El usuario queda con eliminado = false. |

HU-13 | Modificar un usuario

Como: operador del sistema

Quiero: editar los datos de un usuario existente

Para: mantener la información actualizada

Prioridad: Alta

Story Points: 8

Criterios de Aceptación

1. El sistema lista los usuarios activos antes de pedir el ID.
2. Si el ID no corresponde a un usuario activo, se muestra error.
3. Se muestran los valores actuales antes de pedir los nuevos.
4. Dejar un campo en blanco conserva el valor anterior.
5. Si se modifica el mail, verificar que no esté en uso por otro usuario activo.
6. El cambio se persiste correctamente.

HU-14 | Dar de baja un usuario

Como: operador del sistema

Quiero: dar de baja un usuario que ya no utiliza el sistema

Para: ocultarlo sin perder el historial de pedidos

Prioridad: Alta

Story Points: 6

Criterios de Aceptación

1. La baja es lógica: eliminado = true, el registro permanece en la BD.
2. Si el ID no existe o ya está dado de baja, se informa el error.
3. El usuario dado de baja no aparece en ningún listado activo.
4. Se confirma mostrando el nombre completo del usuario afectado.
5. Sus pedidos permanecen en el sistema sin modificación.

HU-15 | Buscar usuario por mail

Como: operador del sistema

Quiero: buscar un usuario ingresando su dirección de mail

Para: consultar rápidamente sus datos sin recorrer el listado

Prioridad: Alta

Story Points: 6

Criterios de Aceptación

1. Se solicita el mail por consola.
2. Se llama a `usuarioRepo.buscarPorMail(mail)`.
3. Si el Optional no está vacío, se muestran todos los datos del usuario (sin mostrar la contraseña).
4. Si el Optional está vacío, se informa que no existe usuario activo con ese mail.

7.5 Pedidos

HU-16 | Dar de alta un pedido con detalles

Como: operador del sistema

Quiero: registrar un pedido completo con sus productos y cantidades

Para: registrar una compra y actualizar el inventario automáticamente

Prioridad: Alta

Story Points: 20

Criterios de Aceptación

1. Se lista y selecciona un usuario activo para asociar al pedido.
2. Se selecciona la forma de pago: TARJETA, TRANSFERENCIA o EFECTIVO.
3. Se muestran los productos activos y disponibles para seleccionar.
4. Por cada producto se valida: que exista, que disponible = true y que tenga stock suficiente.
5. Si alguna validación falla después de iniciar la transacción, se hace rollback completo.
6. El subtotal de cada `DetallePedido` se calcula como `precio * cantidad`.
7. El total del Pedido se calcula llamando a `calcularTotal()` (suma de subtotales).
8. El stock de cada producto se reduce al confirmar el pedido.
9. El estado inicial es PENDIENTE y la fecha es la fecha actual.

10. Toda la operación ocurre dentro de una única transacción atómica.

11. Al guardar se muestra: ID generado, total, usuario y resumen de productos con cantidades y subtotales.

HU-17 | Cambiar estado de un pedido

Como: operador del sistema

Quiero: actualizar el estado de un pedido existente

Para: reflejar el progreso de la orden en el sistema

Prioridad: Alta

Story Points: 8

Criterios de Aceptación

1. Se solicita el ID del pedido.

2. Si no existe o está dado de baja, se informa el error.

3. Se muestra el estado actual del pedido.

4. Se permite seleccionar el nuevo estado: PENDIENTE, CONFIRMADO, TERMINADO o CANCELADO.

5. El cambio se persiste correctamente.

6. Se confirma mostrando el ID del pedido y su nuevo estado.

HU-18 | Dar de baja un pedido

Como: operador del sistema

Quiero: dar de baja un pedido incorrecto o cancelado

Para: ocultarlo del sistema sin perder el historial

Prioridad: Alta

Story Points: 6

Criterios de Aceptación

1. La baja es lógica: eliminado = true, el registro permanece en la BD.

2. Si el ID no existe o ya está dado de baja, se informa el error.
3. El stock de los productos NO se restaura.
4. Los DetallePedido permanecen en la BD.
5. Se confirma mostrando el ID y el total del pedido dado de baja.

HU-19 | Consulta JPQL: Pedidos por usuario

Como: operador del sistema

Quiero: ver todos los pedidos activos de un usuario específico

Para: consultar el historial de compras de un cliente

Prioridad: Alta

Story Points: 8

Criterios de Aceptación

1. Se lista y selecciona un usuario activo.
2. Se llama a `pedidoRepo.buscarPorUsuario(idUsuario)`.
3. Se muestra: ID, fecha, estado, forma de pago y total de cada pedido.
4. Si el usuario no tiene pedidos activos, se informa explícitamente.
5. La consulta usa `List<Pedido>` con parámetro nombrado y `filtra eliminado = false`.

HU-20 | Consulta JPQL: Pedidos por estado

Como: operador del sistema

Quiero: filtrar pedidos activos por su estado

Para: gestionar los pedidos pendientes o terminados rápidamente

Prioridad: Alta

Story Points: 8

Criterios de Aceptación

1. Se muestran las opciones disponibles: PENDIENTE, CONFIRMADO, TERMINADO, CANCELADO.
2. Se llama a `pedidoRepo.buscarPorEstado(estado)`.
3. Se muestra: ID, fecha, nombre de usuario y total de cada pedido.
4. Si no hay pedidos con ese estado, se informa explícitamente.
5. La consulta usa `List<Pedido>` con parámetro nombrado y filtra `eliminado = false`.

HU-21 | Reporte: Total facturado

Como: operador del sistema

Quiero: conocer el total acumulado de los pedidos en estado TERMINADO

Para: tener una vista rápida de la facturación del sistema

Prioridad: Alta

Story Points: 5

Criterios de Aceptación

1. Al seleccionar la opción, el sistema obtiene los pedidos con estado TERMINADO y `eliminado = false`.
2. Suma los campos total de todos los pedidos del resultado.
3. Muestra el resultado formateado (ej: Total facturado: \$12500.00).
4. Si no hay pedidos terminados, muestra \$0.00.

Entrega del Proyecto

Contenido de la Entrega

1. Código Fuente

- Repositorio GitHub con todo el código del frontend y el backend.
- Archivo README.md en la raíz del repositorio con instrucciones claras de instalación, configuración de la base de datos y comandos para ejecutar el proyecto.

2. Documentación Académica y Técnica (Formato PDF)

El documento debe estar correctamente ordenado, con las hojas numeradas y mantener un formato profesional. Debe incluir:

- **Carátula:** Nombre de la institución, carrera, nombre de la materia, título del proyecto ("Food Store"), datos del estudiante y fecha de entrega.
- **Índice:** Tabla de contenidos estructurada con los números de página correspondientes para facilitar la navegación.
- **Marco Teórico: Breve descripción de las tecnologías utilizadas (Java 23, JPA/Hibernate, H2, Vite, etc.) y conceptos clave aplicados.**
- **Decisiones Técnicas y Arquitectura:** Explicación de cómo se abordó la solución, por qué se aplicaron ciertos patrones (por ejemplo, el uso de DTOs, manejo global de excepciones), y capturas de pantalla de las interfaces principales del sistema.
- **Dificultades y Soluciones:** Un breve apartado comentando los principales obstáculos técnicos que surgieron durante los sprints y cómo lograron resolverlos.
- **Bibliografía / Webgrafía:** Enlaces a la documentación oficial, repositorios o recursos utilizados como apoyo durante el desarrollo.

3. Video Demostración

- Grabación de 10 a 15 minutos mostrando el funcionamiento del sistema ya levantado.
- Se debe recorrer obligatoriamente el flujo del cliente (registro, navegación, uso del carrito, confirmación de pedido) y el flujo del administrador (gestión del catálogo y actualización de estados de pedidos).

Método de Entrega:

La entrega del proyecto deberá realizarse **exclusivamente en formato .zip**. El archivo comprimido deberá contener obligatoriamente:

- El **código fuente completo desarrollado**.
- El **documento en formato PDF** confeccionado según las consignas establecidas.

Asimismo, dentro del archivo **README.md** del repositorio se deberá incluir obligatoriamente:

- El **enlace al video demostrativo**, asegurando que cuente con permisos públicos de visualización.
- El **enlace a la documentación en PDF**, o bien el archivo PDF subido directamente en la raíz del repositorio.

No se considerarán entregas incompletas o que no respeten el formato indicado.